

CS61B SP21 项目2: Gitlet 完整中文翻译

部分内容由豆包生成



本文档为 UC Berkeley CS61B 数据结构课程 2021 春季学期 Project 2: Gitlet 的完整规范中文翻译，保留了所有代码示例、示意图、格式要求，可直接作为开发参考。

目录

- 关于本规范的说明
- Gitlet 概述
- 内部数据结构
- 行为详细规范
 - 通用规范
- 命令详解
 - init
 - add
 - commit
 - rm
 - log
 - global-log
 - find
 - status
 - checkout
 - branch
 - rm-branch
 - reset
 - merge
- 项目骨架代码
- 设计文档要求

- 评分标准
 - 检查点评分
 - 完整评分
 - Snaps提交评分
 - 加分项
- 项目通用须知
- 文件处理指南
- 序列化细节
- 测试指南
- 使用官方参考实现测试
- 集成测试详解
 - 测试示例
 - 测试初始化
 - 输出模板匹配
 - 测试总结
- 集成测试调试
 - 定位出错的执行步骤
- 远程功能（加分项）
 - 远程命令详解
- 开发避坑清单
- 致谢

一、关于本规范的说明

本规范篇幅较长，前半部分详细描述了所有需要支持的命令，后半部分是测试细节和开发建议。为了帮助你理解，官方准备了大量高质量视频讲解规范内容和入门指南，所有视频都在对应章节链接，也可以在课程官网查看完整视频列表。

注意：部分视频制作于2020年春季，当时Gitlet是项目3、Capers是实验12，部分视频提到了Hilfinger教授的CS61B配置（包括名为shared的远程仓库、repo仓库等），这些内容本学期无需关注，作业核心要求没有变化。

二、Gitlet 概述

 **前置要求：**开始本项目前请确保你已经完成了实验6：Canine Capers。实验6是本项目的入门引导，会帮助你完成环境配置和基础概念理解。同时建议观看第12讲：Gitlet，其中介绍了本项目的大量核心概念。

在本项目中，你将实现一个模仿流行版本控制系统Git核心功能的版本控制系统，我们的实现更精简小巧，因此命名为Gitlet。

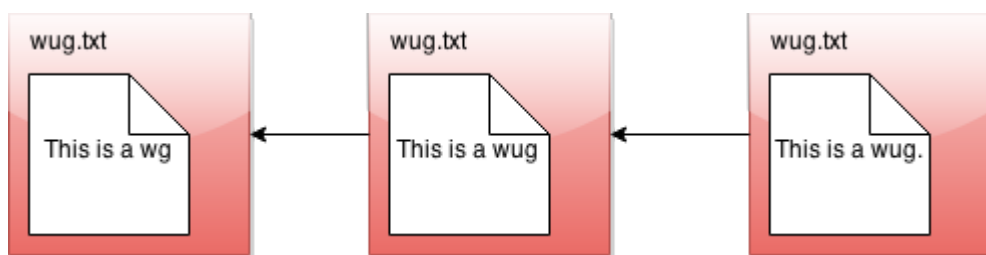
版本控制系统本质上是关联文件集合的备份系统，Gitlet支持的核心功能包括：

1. 保存整个目录下文件的内容：在Gitlet中这个操作称为**提交（committing）**，保存的内容快照称为**提交（commits）**。
2. 恢复一个或多个文件、甚至整个提交的版本：在Gitlet中这个操作称为**检出（checking out）**对应文件或提交。
3. 查看备份历史：在Gitlet中通过**日志（log）**查看历史。
4. 维护关联的提交序列，称为**分支（branches）**。
5. 将一个分支的修改合并到另一个分支。

版本控制系统的意义在于帮助你开发复杂项目（甚至简单项目），或者和他人协作开发项目。你可以定期保存项目版本，如果后续不小心改坏了代码，可以恢复到之前提交的版本（不会丢失之后的修改）；如果协作者提交了修改，你可以将这些修改合并（merge）到自己的版本中。

在Gitlet中，你不需要每次只提交单个文件，而是可以同时提交一组关联的文件。我们可以将每个提交看作是项目在某个时间点的完整**快照（snapshot）**。为了简化说明，本文档的很多示例只会修改单个文件，但请记住每次提交可以修改多个文件。

我们可以将提交随时间的变化可视化：假设项目只有wug.txt一个文件，我们添加一些文本后提交，然后修改文件再次提交，再修改再提交，这样就保存了该文件的3个版本，每个版本时间依次靠后。可视化效果如下：

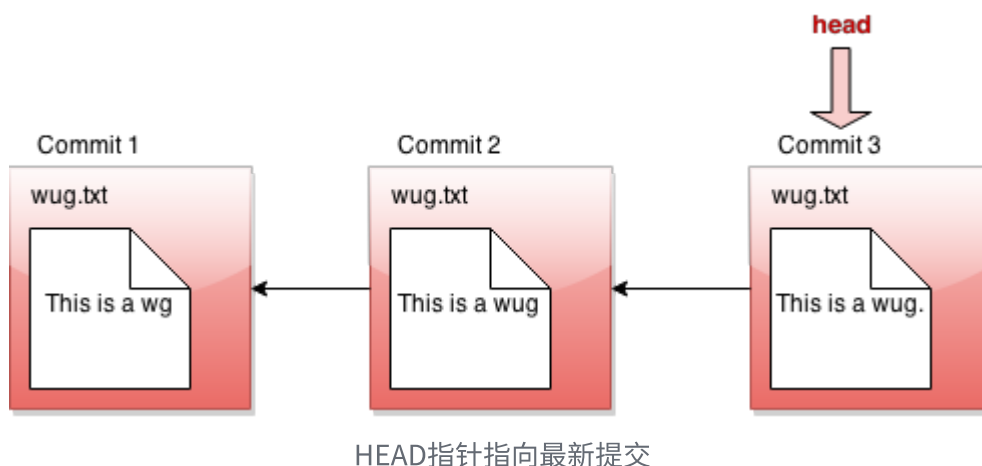


线性提交链示意图

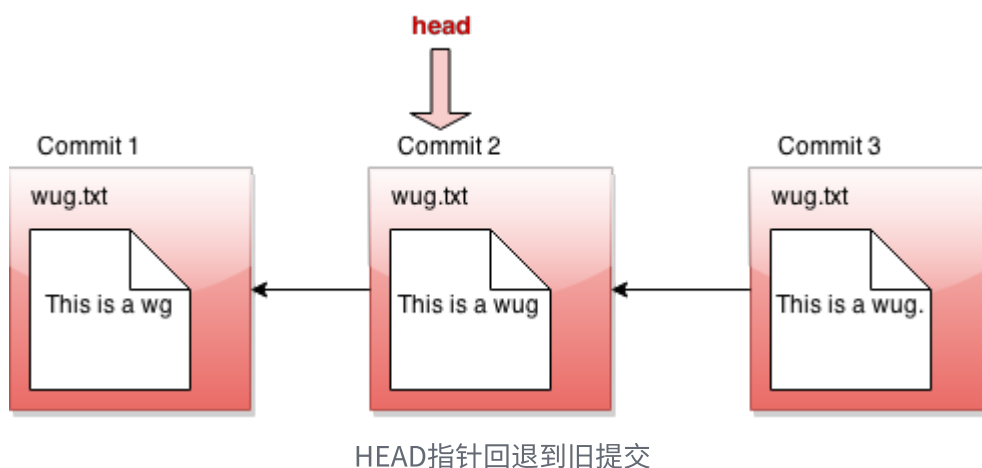
图中的箭头表示每个提交都包含指向前一个提交的引用，我们称前一个提交为**父提交（parent commit）**——这个概念后续会很重要。这个结构是不是很熟悉？没错，就是链表！

Gitlet的核心思想就是用这样的链表结构可视化文件不同版本的历史，这样我们就可以轻松恢复旧版本文件。比如你可以执行命令“Gitlet，请恢复到第2个提交的文件状态”，程序就会回到链表第二个节点，恢复那里保存的文件副本，同时删除第一个节点有但第二个节点没有的文件。

如果我们让Gitlet回退到旧提交，链表头就不再反映当前文件状态，会产生误导。为了解决这个问题，我们引入**头指针（HEAD pointer）**，头指针记录我们当前在链表中的位置。正常提交时，头指针会保持在链表头部，表示最新提交反映当前文件状态：



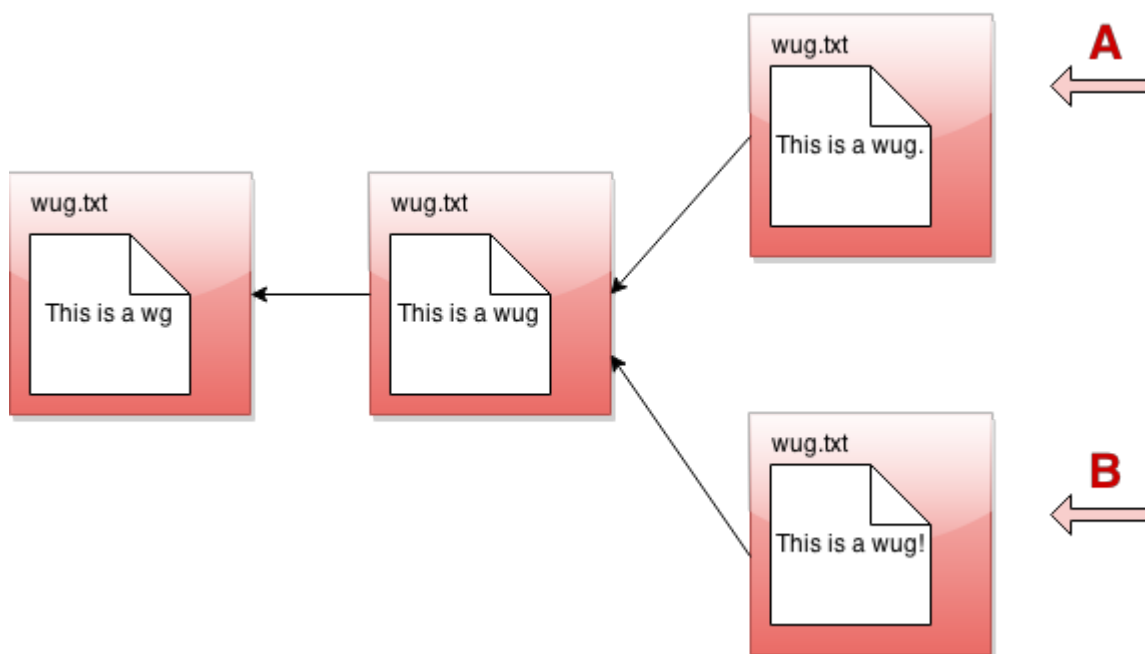
如果我们回退到第2个提交的状态（也就是后续会讲到的reset命令），头指针会向后移动：



这种状态称为**分离头指针状态（detached head state）**，你可能在使用Git时遇到过。

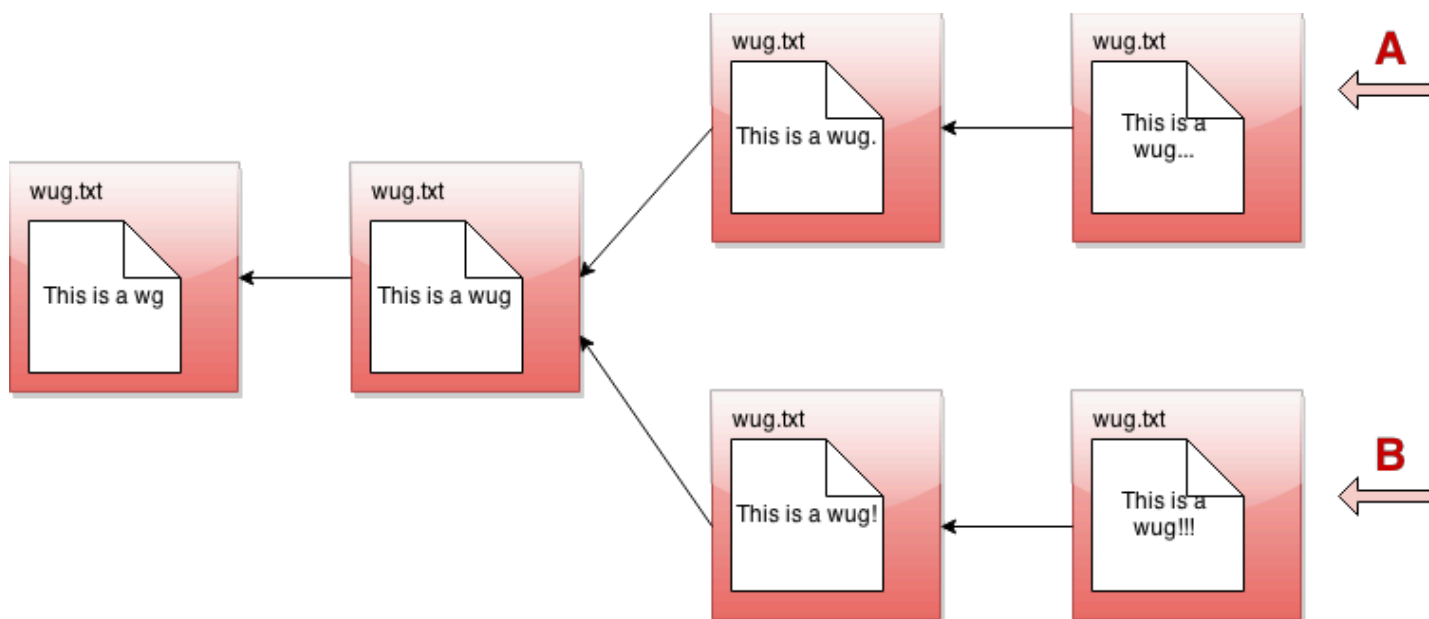
3/5更新：注意在Gitlet中，不存在分离头指针状态，因为没有可以将HEAD移动到任意提交的checkout命令；reset命令虽然会移动HEAD，但同时也会移动分支指针，因此Gitlet中永远不会出现分离HEAD状态。

如果Gitlet只有这些功能，那它会是一个很简单的系统。但Gitlet还有一个核心功能：它不仅可以维护文件的新旧版本，还可以维护**不同的版本**。想象你在开发项目，有两个实现思路：方案A和方案B，Gitlet允许你同时保存两个版本，随时切换。可视化效果如下：



分支分叉示意图

这时候它不再是链表，更像一棵树，我们称之为**提交树（commit tree）**。按照这个比喻，每个独立的版本称为树的一个**分支（branch）**，你可以在每个分支上独立开发：



双分支独立开发示意图

树上有两个指针，分别指向每个分支的最新提交。任意时刻只有一个指针是当前活跃指针，也就是头指针，头指针指向当前分支的头部。

以上就是Gitlet系统的简要概述！如果你还没完全理解也不用担心，上面的内容只是高层概念介绍，后续会有详细的规范说明。

最后一点：提交树是**不可变（immutable）**的：一旦提交节点创建，就永远不能被删除或修改，我们只能向提交树添加新节点，不能修改已有节点。这是Gitlet的重要特性，目的就是保证保存的内容不会被意外删除。

三、内部数据结构

真实Git区分多种不同的**对象（objects）**，对我们来说重要的对象包括：

- **blob（二进制对象）**：保存的文件内容。因为Gitlet会保存文件的多个版本，单个文件可能对应多个blob，每个blob在不同提交中被跟踪。
- **tree（树对象）**：目录结构，将文件名映射到blob和其他tree（子目录）的引用。
- **commit（提交对象）**：组合了提交日志、其他元数据（提交时间、作者等）、tree的引用、父提交的引用。仓库还维护**分支头（branch heads）**到提交引用的映射，让重要的提交拥有符号名称。

Gitlet相比Git做了进一步简化：

- 将tree合并到commit中，不处理子目录（每个仓库只有一层平铺的普通文件）。
- 限制合并提交只能有两个父提交（真实Git可以有任意数量父提交）。
- 元数据只包含时间戳和提交日志。因此一个提交包含：提交日志、时间戳、文件名到blob引用的映射、父提交引用，合并提交还包含第二个父提交引用。

每个对象——我们实现中的每个blob和每个commit——都有唯一的整数ID作为对象引用。Git的一个有趣特性是这些ID是**全局通用**的：和典型Java实现不同，内容完全相同的两个对象在所有系统上都会有相同的ID（你的电脑、我的电脑、任何人的电脑计算出的ID都完全一致）。对于blob，“内容相同”指文件内容完全一致；对于commit，指元数据、文件名到引用的映射、父提交引用完全一致。因此仓库中的对象称为**内容寻址（content addressable）**。

Git和Gitlet都通过相同方式实现这一点：使用名为SHA-1（安全哈希算法1）的**加密哈希函数**，可以从任意字节序列生成160位整数哈希。加密哈希函数的特性是：极难找到两个不同字节序列拥有相同哈希值（实际上也极难仅通过哈希值反推出原始字节序列），因此我们可以认为不同内容对象拥有相同SHA-1哈希的概率是 2^{-160} ，约 10^{-48} 。我们直接忽略哈希冲突的可能性，虽然理论上这是系统bug，但实际中永远不会发生！

幸运的是，有现成的库类可以计算SHA-1值，你不需要自己实现算法，只需要正确标记所有对象即可，具体包括：

- 计算commit哈希时包含所有元数据和引用。
- 通过某种方式区分commit哈希和blob哈希：好的实现方式是在.gitlet目录下设计合理的目录结构；另一种方式是计算哈希时为每个对象额外加入一个标识字，blob和commit使用不同值。

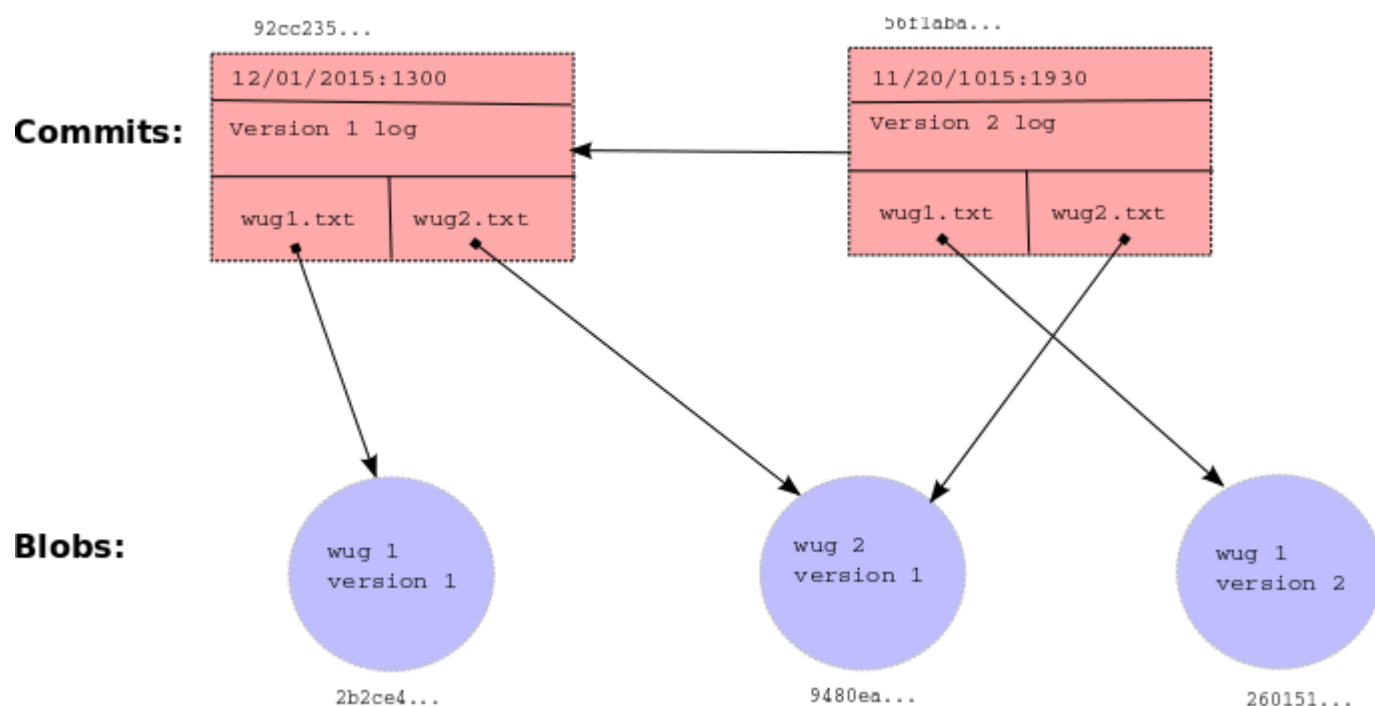
另外，SHA-1哈希值表示为40位十六进制字符串，可以很方便地作为.gitlet目录下存储数据的文件名（后续会详细说明）。同时它也提供了比较两个文件（blob）内容是否相同的便捷方式：如果SHA-1相同，我们就认为文件相同。

对于远程功能（比如我们整学期使用的skeleton远程仓库），我们直接使用其他Gitlet仓库。推送（push）就是将远程仓库没有的提交和blob复制到远程，然后更新分支引用；拉取（pull）方向相反。远程功能是本项目的加分项，满分不需要实现。

得益于Java的**序列化（serialization）**功能，从文件读写内部对象其实很简单。

`java.io.Serializable` 接口没有任何方法，但如果一个类实现了该接口，Java运行时会自动提供对象和字节流的双向转换能力，你可以使用 `java.io.ObjectOutputStream` 将对象写入文件，使用 `java.io.ObjectInputStream` 读取（反序列化）对象。“序列化”指将任意结构（数组、树、图等）转换为连续字节序列的过程。你在实验6中已经练习过序列化，本项目使用方法非常类似，实现持久化和序列化时可以参考实验6的代码。

下图是本节讨论的结构示例：可以看到每个commit（矩形）指向一些blob（圆形），blob包含文件内容。commit包含文件名和对应blob的引用，以及父节点链接。这些引用（图中箭头）在.gitlet目录中使用SHA-1哈希值表示（commit上方和blob下方的十六进制数字）。较新的commit包含更新版本的wug1.txt，但和旧commit共享相同版本的wug2.txt。你的Commit类需要存储图中展示的所有信息：精心选择内部数据结构会让实现更简单，因此花时间规划最优存储方式是很有必要的。



commit和blob结构示意图

四、行为详细规范

4.1 通用规范

我们唯一的结构要求是：你必须有一个名为 `gitlet.Main` 的类，且该类包含main方法。

我们提供了一些工具方法，主要用于处理文件系统相关任务，让你可以专注于项目逻辑，而不用处理操作系统的特殊细节。

我们还提供了两个建议类：`Commit` 和 `Repository` 帮助你入门。你当然可以编写额外的Java类支持项目，也可以删除我们提供的建议类，但**不能修改Main类**，否则自动判分程序无法找到你的代码。不要使用任何外部代码（JUnit除外），只能使用Java语言，可以使用任意Java标准库类和我们提供的工具类。

 **不要把所有逻辑都写在Main类中。** Main类应该主要调用Repository类中的辅助方法，可以参考实验6中的CapersRepository和Main类的结构，这是我们推荐的代码结构。

本规范大部分内容会描述 `Gitlet.java` 的main方法接收到不同gitlet命令参数时的行为。在逐个讲解命令前，先说明整个项目需要满足的通用规则：

- Gitlet正常运行需要一个地方存储文件旧副本和其他元数据，所有这些数据**必须**存储在名为 `.gitlet` 的目录中，就像真实Git将同类信息存储在 `.git` 目录一样（文件名以.开头是隐藏文件，大多数操作系统默认不显示，Unix系统中执行 `ls -a` 可以查看隐藏文件）。如果某个位置存在.gitlet目录，就认为该位置已经初始化了Gitlet系统。除了init命令外，大多数Gitlet命令只需要在已初始化Gitlet的目录中工作——也就是包含.gitlet子目录的目录。不在.gitlet目录中的文件（你正在编辑的仓库文件副本、你准备添加到仓库的文件）统称为**工作目录（working directory）**文件。
- 大多数命令有运行时间或内存使用要求，你必须遵守。部分运行时间描述为“相对于任何关键指标为常数时间”，关键指标包括：文件数量/大小、提交数量。序列化和反序列化的时间可以忽略，但有一个限制：序列化时间**绝对不能**随已添加、已提交文件的总大小变化而变化（如果你不知道序列化是什么，请复习实验6）。你可以认为哈希表的存取操作是常数时间。
- 部分命令有指定的失败场景和固定错误信息，具体格式后续会说明。所有错误信息必须以英文句号结尾，因为自动判分会逐字匹配输出，请确保包含句号。如果程序遇到任何指定失败场景，必须打印错误信息，且**不能修改任何其他内容**。除了列出的失败场景外，你不需要处理其他错误情况。
- 有一些通用失败场景适用于所有命令，不针对单个命令：
 - 用户没有输入任何参数：输出 `Please enter a command.` 并退出。
 - 用户输入不存在的命令：输出 `No command with that name exists.` 并退出。
 - 用户输入命令的操作数数量错误或格式错误：输出 `Incorrect operands.` 并退出。
 - 用户执行需要在已初始化Gitlet目录中运行的命令，但当前目录没有.gitlet子目录：输出 `Not in an initialized Gitlet directory.` 并退出。
- 部分命令列出了和真实Git的区别，本规范不会列出所有差异，但会列出影响较大、容易造成混淆和误导的主要差异。
- **禁止输出规范以外的任何内容**，多余打印会导致自动判分测试失败。
- 如果需要立即退出程序，可以调用 `System.exit(0)`。比如辅助函数中途发生错误需要立即终止整个Gitlet程序时就可以调用该方法。注意：`System.exit()` 的参数必须固定为0，CS61C课程中会讲解这个参数（错误码）的含义。
- 规范将部分命令标记为“危险命令”：危险命令指可能覆盖普通文件（而不仅仅是元数据）的命令。比如用户让Gitlet将文件恢复到旧版本时，Gitlet可能会覆盖当前目录中的文件版本。仅作参考，测试这些命令前请做好备份。

五、命令详解

接下来我们详细讲解每个需要支持的命令。记住优秀的程序员会重视数据结构设计：阅读这些命令时，你应该首先思考如何存储数据才能轻松支持这些命令，其次思考是否有机会复用已经实现的命令（提示：本项目有大量机会复用前面写的代码）。部分命令列出了推荐参考的课程讲座，但你不需要必须使用这些讲座中的概念。部分容易混淆的命令配有概念测验，建议你用来检查自己的理解，测验不计分，只是帮助你在实现命令前确认理解正确。

5.1 init 初始化

- 用法：

代码块

```
1 java gitlet.Main init
```

- **功能：**在当前目录创建新的Gitlet版本控制系统。系统会自动创建一个初始提交：该提交不包含任何文件，提交信息固定为 `initial commit`（没有标点）。初始只有一个分支 `master`，初始指向这个初始提交，`master`为当前分支。初始提交的时间戳固定为UTC时间1970年1月1日 00:00:00（称为Unix纪元，内部用时间0表示），你可以自行选择日期格式。因为所有Gitlet仓库的初始提交内容完全相同，因此所有仓库会共享这个初始提交（拥有相同UID），仓库中所有提交最终都可以追溯到这个初始提交。
- **运行时间：**相对于任何关键指标为常数时间。
- **失败场景：**如果当前目录已经存在Gitlet版本控制系统，终止命令，不能覆盖已有系统，输出错误信息：

代码块

```
1 A Gitlet version-control system already exists in the current directory.
```

- **危险命令？：**否
- **参考代码行数：**约15行

5.2 add 添加文件到暂存区

- 用法：

代码块

```
1 java gitlet.Main add [文件名]
```

- **功能：**将文件当前版本的完整副本添加到**暂存区（staging area）**（参考commit命令说明），因此添加文件也称为“暂存文件待提交”。如果文件已经在暂存区，会用新内容覆盖暂存区中的旧条目。暂存区应该位于.gitlet目录内部。如果当前工作目录的文件版本和当前提交中的版本完全相同，则不要将其加入暂存区，如果它已经在暂存区则移除（典型场景：文件修改后add，又改回原始版本）。如果文件之前被标记为待删除（参考rm命令），执行add后会取消待删除标记。
- **运行时间：**最坏情况下，与被添加文件的大小线性相关，与当前提交中文件数量N的对数lgN线性相关。
- **失败场景：**文件不存在时输出错误信息：

代码块

```
1 File does not exist.
```

- **危险命令？：**否
- **参考代码行数：**约20行
- **和真实Git的区别：**真实Git支持一次add多个文件，Gitlet一次只能添加一个文件。
- **推荐讲座：**第16讲（集合、映射、抽象数据类型）、第19讲（哈希）

5.3 commit 提交

- **用法：**

代码块

```
1 java gitlet.Main commit [提交信息]
```

- **功能：**保存当前提交和暂存区中被跟踪文件的快照，以便后续恢复，同时创建新提交。该提交称为**跟踪（tracking）**这些保存的文件。默认情况下，每个提交的文件快照和父提交的文件快照完全相同，会保持文件版本不变，不会更新。提交只会更新那些在提交时被标记为待添加暂存的跟踪文件，此时提交会使用暂存区中的文件版本，替换从父提交继承的版本。对于暂存待添加但父提交没有跟踪的文件，提交会保存并开始跟踪它们。最后，当前提交跟踪的文件如果被rm命令标记为**待删除（staged for removal）**，在新提交中会取消跟踪。总结：默认提交和父提交文件内容完全相同，只有待添加和待删除暂存的文件是提交的更新内容，当然提交时间（通常还有提交信息）也会和父提交不同。commit额外说明：
 - 提交后清空暂存区。
 - commit命令永远不会添加、修改、删除工作目录中的文件（除了.gitlet目录内的文件）。rm命令会删除工作目录文件，并标记为待删除，这样commit后这些文件就会被取消跟踪。

- 文件被标记为待添加或待删除暂存后，对工作目录文件的任何修改都会被commit命令忽略，commit只会修改.gitlet目录的内容。比如你用Unix的rm命令（而不是Gitlet的rm命令）删除了一个被跟踪的文件，对下一次commit没有影响，提交中仍然会包含这个（已经被删除的）文件版本。
- commit命令执行后，新提交会作为新节点添加到提交树中。
- 刚创建的提交成为“当前提交”，头指针指向它，之前的头提交是这个提交的父提交。
- 每个提交必须包含创建的日期和时间。
- 每个提交关联一段用户提供的日志描述，说明提交中的文件修改。提交信息作为main方法args数组的单独一项传入，多词信息需要用引号包裹。
- 每个提交通过SHA-1 ID标识，哈希计算必须包含：文件（blob）引用、父提交引用、提交日志、提交时间。
- **运行时间与内存：**运行时间相对于提交数量为常数时间，相对于提交跟踪的所有文件总大小不超过线性时间。此外该命令有内存要求：commit后.gitlet目录增加的大小不能超过提交时待添加暂存文件的总大小（不包含额外元数据）。这意味着不要重复存储从父提交继承的文件版本（提示：记住blob是内容寻址的，利用SHA-1特性）。你可以存储文件的完整副本，不需要只存储差异diff。
- **失败场景：**如果没有暂存任何文件，终止并输出：

代码块

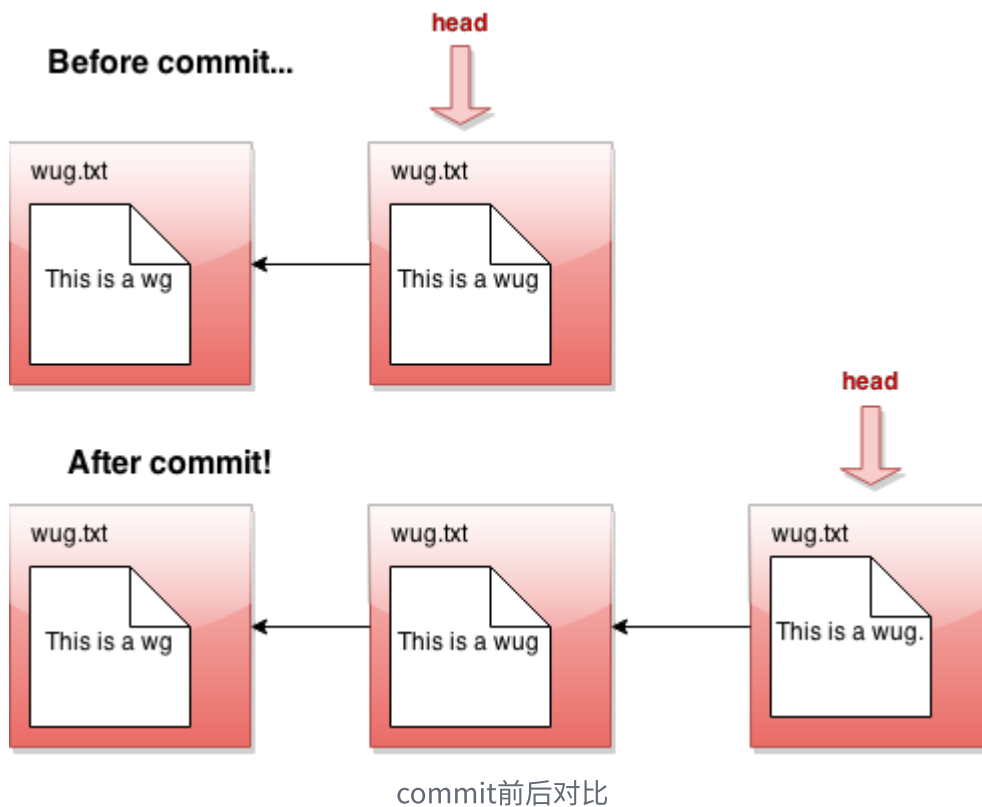
```
1  No changes added to the commit.
```

代码块

```
1  Please enter a commit message.
```

- **危险命令？：**否
- **和真实Git的区别：**真实Git的提交可以有多个父提交（因为合并），并且包含更多元数据。
- **参考代码行数：**约35行
- **推荐讲座：**第16讲（集合、映射、抽象数据类型）、第19讲（哈希）

commit前后示意图：



5.4 rm 删除文件

- 用法：

代码块

```
1 java gitlet.Main rm [文件名]
```

- **功能：**如果文件当前被标记为待添加暂存，取消其暂存状态。如果文件在当前提交中被跟踪，将其标记为待删除，如果用户还没有删除工作目录中的该文件则删除它（只有文件被当前提交跟踪时才删除，否则不删除）。
- **运行时间：**相对于任何关键指标为常数时间。
- **失败场景：**文件既没有被暂存待添加，也没有被头提交跟踪，输出错误：

代码块

```
1 No reason to remove the file.
```

- **危险命令？：**是（如果你使用我们提供的工具方法，只会影响仓库文件，不会误删目录中其他无关文件）
- **参考代码行数：**约20行

5.5 log 查看当前分支日志

- 用法：

代码块

```
1 java gitlet.Main log
```

- **功能：**从当前头提交开始，沿着提交树反向显示每个提交的信息，直到初始提交，只跟随第一父提交链接，忽略合并提交中的第二父提交（等价于真实Git的 `git log --first-parent`）。这组提交节点称为提交的**历史（history）**。历史中的每个节点需要显示：提交ID、提交时间、提交信息。输出严格遵循以下格式：

代码块

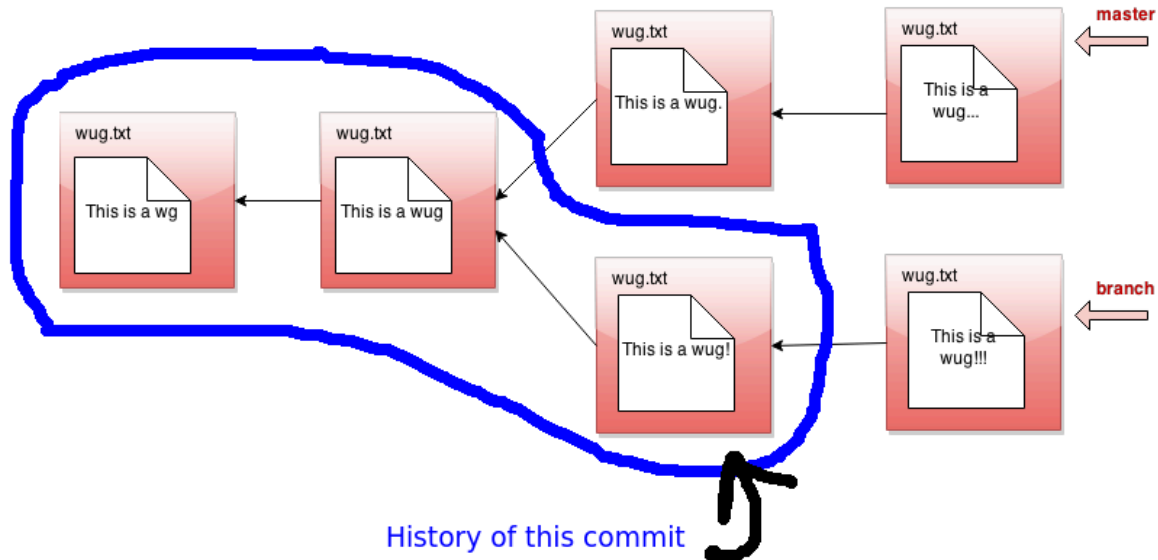
```
1  ===
2  commit a0da1ea5a15ab613bf9961fd86f010cf74c7ee48
3  Date: Thu Nov 9 20:00:05 2017 -0800
4  A commit message.
5
6  ===
7  commit 3e8bf1d794ca2e9ef8a4007275acf3751c7170ff
8  Date: Thu Nov 9 17:01:33 2017 -0800
9  Another commit message.
10
11 ===
12 commit e881c9575d180a215d1a63645b8fd9abfb1d2bb
13 Date: Wed Dec 31 16:00:00 1969 -0800
14 initial commit
15
```

代码块

```
1  ===
2  commit 3e8bf1d794ca2e9ef8a4007275acf3751c7170ff
3  Merge: 4975af1 2c1ead1
4  Date: Sat Nov 11 12:30:00 2017 -0800
5  Merged development into master.
6
```

- **运行时间：**与头提交历史中的节点数量线性相关。
- **失败场景：**无
- **危险命令？：**否
- **参考代码行数：**约20行

下图展示了某个提交的历史范围，如果当前分支头指针指向该提交，log会打印圈出的提交信息：



提交历史范围示意图

历史会忽略其他分支和未来的提交。现在有了历史概念，我们可以进一步说明提交树不可变的含义：不可变指的是**固定ID提交的历史永远不会改变**。如果将提交树看作是多个历史的集合，本质上就是说每个历史都是不可变的。

5.6 global-log 查看所有提交日志

- 用法：

代码块

```
1 java gitlet.Main global-log
```

- **功能：**和log类似，但显示仓库中所有曾经创建过的提交信息，提交顺序不做要求。提示：gitlet.Utils中有遍历目录下文件的实用方法。
- **运行时间：**与仓库总提交数量线性相关。
- **失败场景：**无
- **危险命令？：**否
- **参考代码行数：**约10行

5.7 find 按提交信息查找

- 用法：

代码块 `java gitlet.Main find [提交信息]`

- **功能：**打印所有提交描述匹配给定信息的提交ID，每行一个，多个匹配则分行输出。提交信息是单个操作数，多词信息需要用引号包裹，规则和commit命令一致。提示：该命令的提示和global-log相同。
- **运行时间：**与提交数量线性相关。
- **失败场景：**没有匹配的提交时输出：

代码块

```
1 Found no commit with that message.
```

- **危险命令？：**否
- **和真实Git的区别：**真实Git没有该命令，类似效果可以通过过滤log输出实现。
- **参考代码行数：**约15行

5.8 status 查看状态

- **用法：**

代码块

```
1 java gitlet.Main status
```

- **功能：**显示当前存在的所有分支，当前分支前标记*，同时显示已暂存待添加和待删除的文件。输出严格遵循以下格式：

代码块

```
1  === Branches ===
2  *master
3  other-branch
4
5  === Staged Files ===
6  wug.txt
7  wug2.txt
8
9  === Removed Files ===
10 goodbye.txt
11
12 === Modifications Not Staged For Commit ===
```

```
13  junk.txt (deleted)
14  wug3.txt (modified)
15
16  === Untracked Files ===
17  random.stuff
18
19
```

- 被当前提交跟踪，工作目录中已修改，但未暂存；
- 已暂存待添加，但工作目录中内容和暂存区不同；
- 已暂存待添加，但工作目录中已被删除；
- 未标记为待删除，但被当前提交跟踪，且工作目录中已被删除。
- **运行时间：**仅依赖于工作目录数据量、待添加/待删除暂存文件数量、分支数量。
- **失败场景：**无
- **危险命令？：**否
- **参考代码行数：**约45行

5.9 checkout 检出

checkout是通用命令，根据参数不同有三种用法，下面分别说明：

- **用法：**

a.

代码块

```
1  java gitlet.Main checkout -- [文件名]
```

b.

代码块

```
1  java gitlet.Main checkout [提交ID] -- [文件名]
```

c.

代码块

```
1  java gitlet.Main checkout [分支名]
```

- **功能：**

- a. 取出头提交中该文件的版本，放入工作目录，覆盖已存在的同名文件，新版本不会被暂存。
- b. 取出指定ID提交中该文件的版本，放入工作目录，覆盖已存在的同名文件，新版本不会被暂存。
- c. 取出给定分支头提交中的所有文件，放入工作目录，覆盖已存在的同名文件；当前分支跟踪但目标分支不存在的文件会被删除；命令结束后，给定分支成为当前分支（HEAD指向它）；除非检出的分支就是当前分支，否则清空暂存区（参考失败场景）。

- **运行时间：**

- a. 与被检出文件大小线性相关。
- b. 与提交快照中所有文件总大小线性相关，相对于提交数量、分支数量为常数时间。

- **失败场景：**

- a. 文件在之前的提交中不存在，终止并输出：

代码块

```
1 File does not exist in that commit.
```

- b. 给定ID的提交不存在，输出：

代码块

```
1 No commit with that id exists.
```

- c. 给定名称的分支不存在，输出：

代码块

```
1 No such branch exists.
```

代码块

```
1 No need to checkout the current branch.
```

代码块

```
1 There is an untracked file in the way; delete it, or add and commit it first.
```

代码块

```
1 a0da1ea5a15ab613bf9961fd86f010cf74c7ee48
```

代码块

```
1 a0da1e
```

- **和真实Git的区别：** 真实Git不会清空暂存区，且会将检出的文件加入暂存；同时真实Git不会执行会覆盖或撤销已暂存修改（添加或删除）的检出操作。
- **危险命令？：** 是！
- **参考代码行数：**
 - a. 约15行
 - b. 约5行
 - c. 约15行

5.10 branch 创建分支

- **用法：**

代码块

```
1 java gitlet.Main branch [分支名]
```

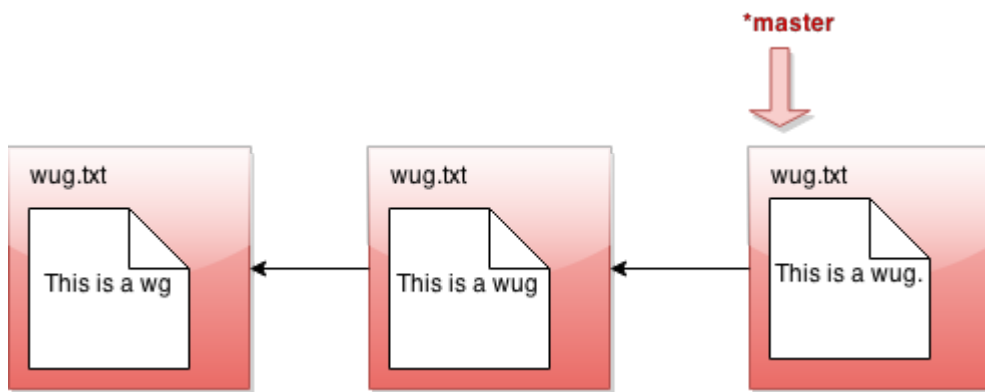
- **功能：** 创建给定名称的新分支，指向当前头提交。分支本质上只是提交节点引用（SHA-1 ID）的命名，该命令**不会立即切换到新创建的分支**，和真实Git行为一致。调用branch命令前，代码应该已经运行在默认分支master上。
- **运行时间：** 相对于任何关键指标为常数时间。
- **失败场景：** 同名分支已存在，输出：

代码块

```
1 A branch with that name already exists.
```

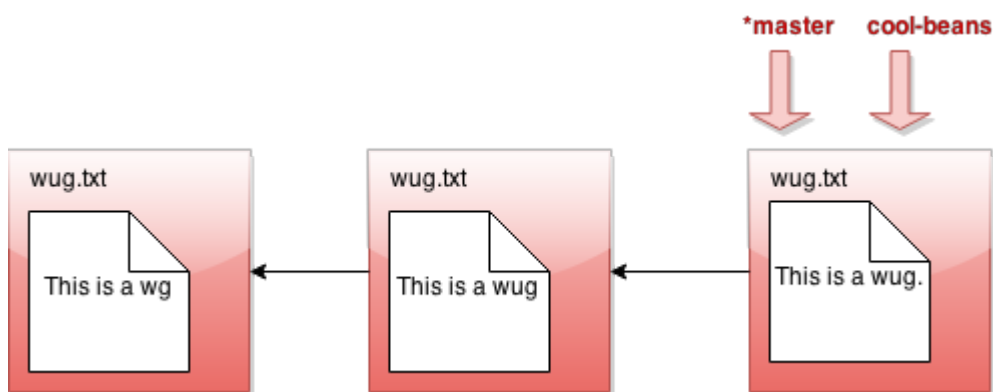
- **危险命令？：** 否
- **参考代码行数：** 约10行

我们来详细看branch的行为：假设初始状态如下：



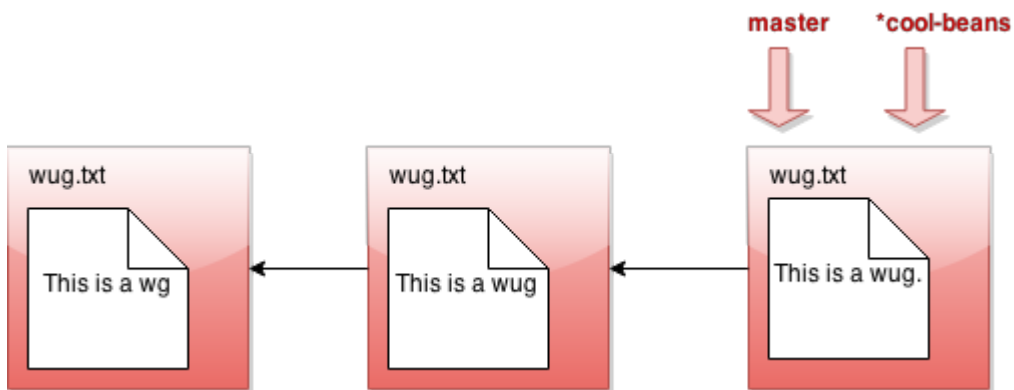
初始状态，只有master分支

执行 `java gitlet.Main branch cool-beans` 后状态变为：



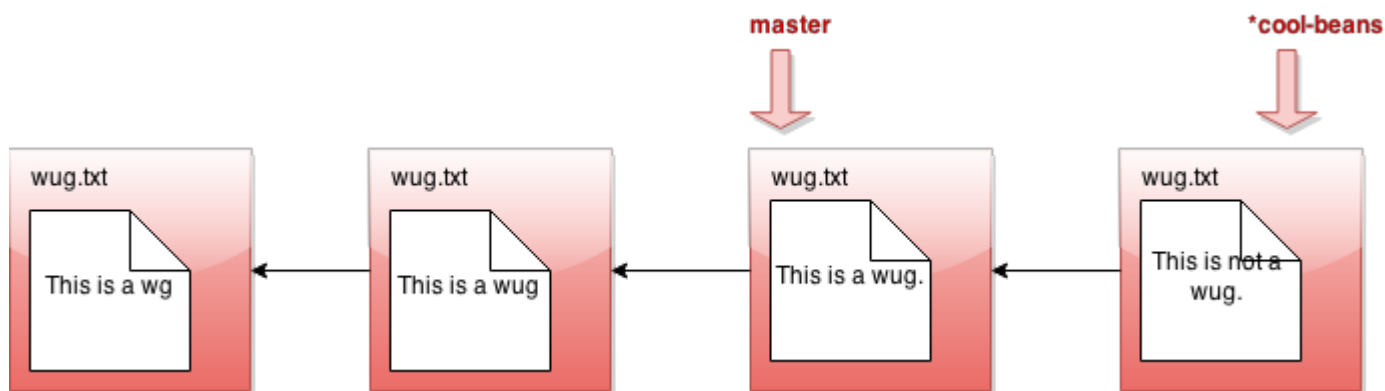
创建cool-beans分支后

看起来没什么变化，执行 `java gitlet.Main checkout cool-beans` 切换分支：



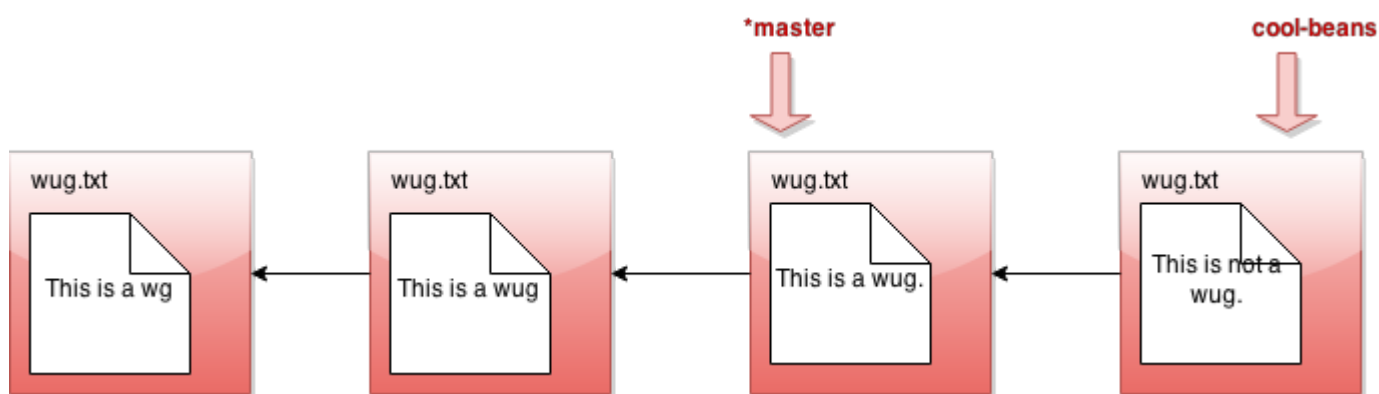
切换到cool-beans分支后

看起来还是没变化！现在我们修改文件，执行add然后commit：



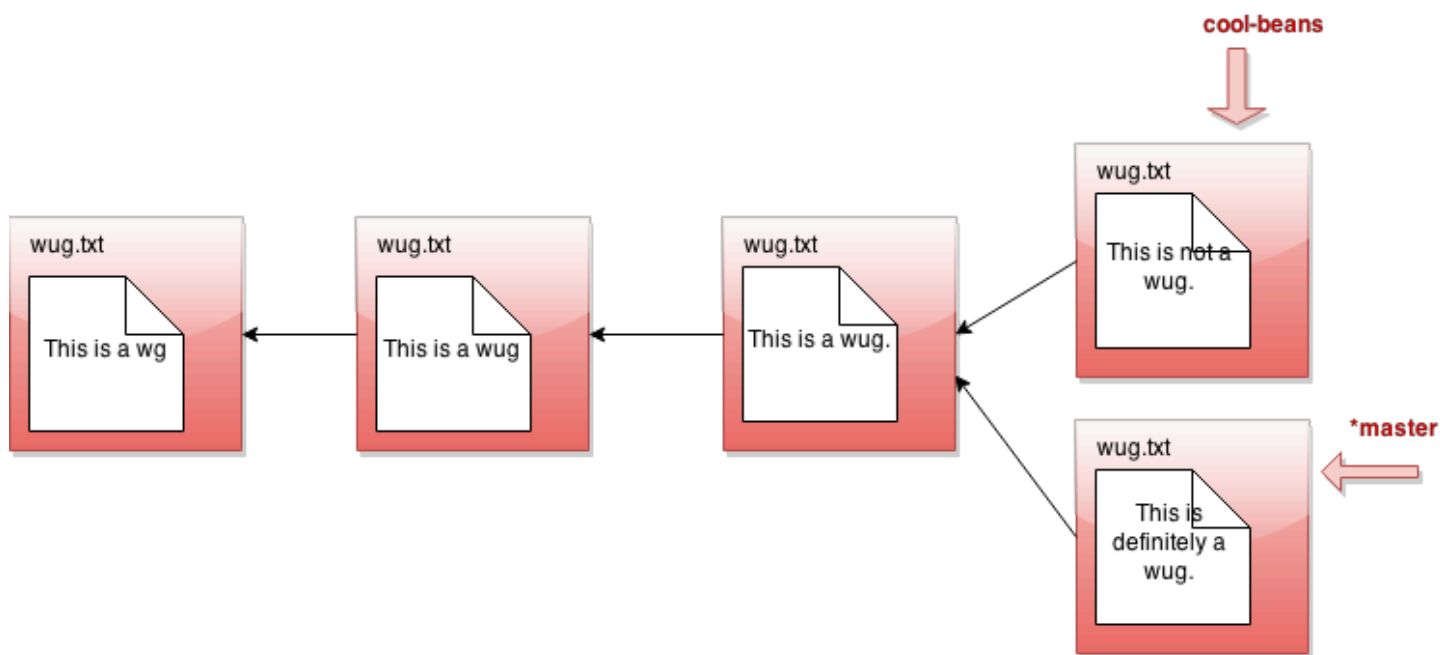
cool-beans分支提交后

说好的分支呢？怎么还是一条直线？我们切回master分支：`java gitlet.Main checkout master`



切回master分支后

现在我们要做一次提交：



master分支提交后，产生分叉

这就是分支的完整原理！你看懂发生了什么吗？创建分支只是给我们一个新的指针，任意时刻其中一个指针是当前活跃指针，也就是HEAD指针（用*标记），我们可以通过`checkout [分支名]`切换活跃头

指针。每次commit时，都会在当前活跃HEAD提交上添加一个子提交，即使该提交已经有子提交，这自然就产生了分支行为，因为一个提交可以有多个子提交。

请确保你的branch、checkout、commit行为和上述描述一致，这是Gitlet的核心功能，很多其他命令都依赖这些功能，如果核心功能出错，大量自动判分测试都会失败。

5.11 rm-branch 删除分支

- 用法：

代码块

```
1 java gitlet.Main rm-branch [分支名]
```

- **功能：**删除给定名称的分支，这并不意味着删除和分支关联的指针，不会删除该分支下创建的所有提交。
- **运行时间：**相对于任何关键指标为常数时间。
- **失败场景：**给定名称的分支不存在，终止并输出：

代码块

```
1 A branch with that name does not exist.
```

代码块

```
1 Cannot remove the current branch.
```

- **危险命令？：**否
- **参考代码行数：**约15行

5.12 reset 重置

- 用法：

代码块

```
1 java gitlet.Main reset [提交ID]
```

- **功能：**检出给定提交跟踪的所有文件，删除该提交中不存在的跟踪文件，同时将当前分支的头移动到该提交节点，参考概述中reset后头指针变化的示例。提交ID和checkout一样支持缩写，清空暂存区。该命令本质上是检出任意提交，同时修改当前分支头。

- **运行时间：**与给定提交快照跟踪的所有文件总大小线性相关，相对于提交数量为常数时间。
- **失败场景：**给定ID的提交不存在，输出：

代码块

```
1  No commit with that id exists.
```

代码块

```
1  There is an untracked file in the way; delete it, or add and commit it
   first.
```

- **危险命令？：**是！
- **和真实Git的区别：**该命令最接近真实Git的 `git reset --hard [提交哈希]`。
- **参考代码行数：**约10行（提示：复用已有代码可以大幅减少代码量）

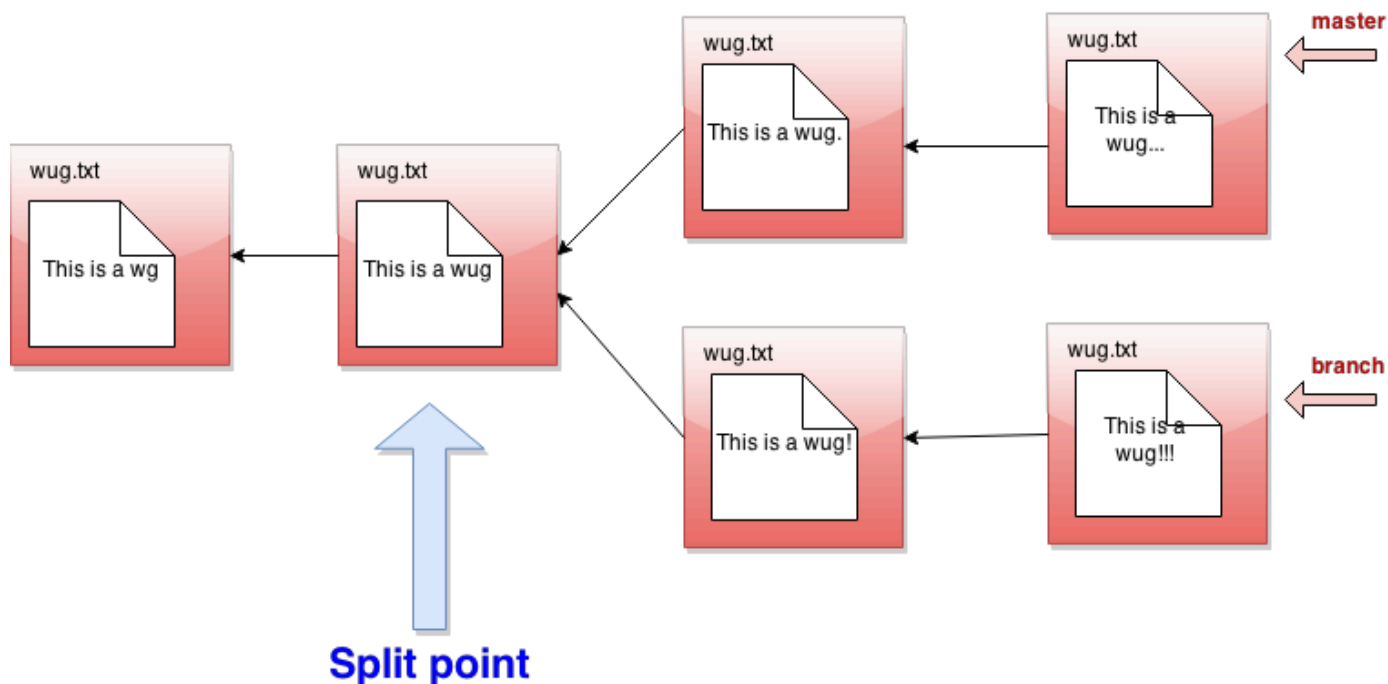
5.13 merge 合并

- **用法：**

代码块

```
1  java gitlet.Main merge [分支名]
```

- **功能：**将给定分支的文件合并到当前分支，该方法逻辑较复杂，详细说明如下：首先考虑当前分支和给定分支的**分叉点（split point）**，比如master是当前分支，branch是给定分支：



分叉点示意图

- **公共祖先**：从两个分支头出发，沿着父指针（0步或多步）都能到达的提交。
- **最近公共祖先**：是公共祖先，且不是其他公共祖先的后代。比如上图最左侧的提交虽然是 master 和 branch 的公共祖先，但它也是它右侧紧邻提交的祖先，因此不是最近公共祖先。

如果分叉点就是给定分支的头提交：什么都不做，合并完成，输出：

代码块

```
1 Given branch is an ancestor of the current branch.
```

如果分叉点是当前分支的头提交：效果等价于检出给定分支，输出：

代码块

```
1 Current branch fast-forwarded.
```

- 给定分支自分叉点后修改、但当前分支自分叉点后未修改的文件，更新为给定分支的版本（从给定分支头提交检出），这些文件自动加入暂存区。“给定分支自分叉点后修改”指给定分支头提交中的文件内容和分叉点版本不同，记住blob是内容寻址的！
- 当前分支自分叉点后修改、但给定分支自分叉点后未修改的文件，保持原样。
- 当前分支和给定分支以相同方式修改的文件（内容相同或都被删除），合并后保持不变。如果文件在两个分支都被删除，但工作目录中存在同名文件，保持文件原样，合并后不跟踪也不暂存。
- 分叉点不存在、仅存在于当前分支的文件，保持原样。

- e. 分叉点不存在、仅存在于给定分支的文件，检出并加入暂存区。
- f. 分叉点存在、当前分支未修改、给定分支已删除的文件，删除并取消跟踪。
- g. 分叉点存在、给定分支未修改、当前分支已删除的文件，保持缺失状态。
- h. 当前分支和给定分支以不同方式修改的文件存在**冲突**。“不同方式修改”包括：两边内容修改且不同、一边修改另一边删除、分叉点不存在但两边内容不同。此时将冲突文件内容替换为：

代码块

```
1 <<<<<<< HEAD
2 当前分支文件内容
3 =====
4 给定分支文件内容
5 >>>>>>>
6
```

代码块

```
1 <<<<<<< HEAD
2 当前分支文件内容=====
3 给定分支文件内容>>>>>>>
4
```

按照上述规则更新文件后，如果分叉点既不是当前分支也不是给定分支，merge会自动提交，提交信息固定为：`Merged [给定分支名] into [当前分支名]`。如果合并遇到冲突，在终端打印提示：

代码块

```
1 Encountered a merge conflict.
```

- **运行时间：** $O(N \lg N + D)$ ，其中N是两个分支所有祖先提交的总数，D是这些提交下所有文件的数据总量。
- **失败场景：**存在暂存的添加或删除操作，输出：

代码块

```
1 You have uncommitted changes.
```

代码块

```
1 A branch with that name does not exist.
```

代码块

```
1 Cannot merge a branch with itself.
```

代码块

```
1 There is an untracked file in the way; delete it, or add and commit it
  first.
```

- **危险命令?**：是!
- **和真实Git的区别**：真实Git的文件合并更精细，只在两边自分叉点后都修改的代码段显示冲突；真实Git选择多个可能分叉点的算法不同；真实Git会强制用户解决所有合并冲突后才能提交完成合并，Gitlet会直接提交包含冲突的合并结果，你需要单独提交来解决问题；真实Git如果存在会被合并修改的文件有未暂存修改会报错，你也可以实现该检查，但我们不会测试该场景。
- **参考代码行数**：约70行
- **推荐讲座**：第19讲（集合、映射、抽象数据类型）、第22讲（图遍历）

六、项目骨架代码

提供的骨架代码非常精简，大部分是空的，我们提供了有用的JavaDoc注释提示每个文件中应该包含什么内容。**你应该采用和Capers类似的结构**：Main类本身不做太多工作，只是根据args调用其他方法。你完全可以删除其他类或添加自己的类，但**必须保留Main类**，否则测试无法找到你的代码。

如果你不知道从何开始，建议复习实验6：Canine Capers。

七、设计文档要求

本次项目没有提供完整骨架，**要求所有人提交一份设计文档描述你的实现思路**。设计文档不计分，但在办公时间答疑或提交Gitbug前，你必须有一份更新完成的设计文档，否则我们无法为你提供帮助。这对双方都有好处：通过写设计文档，你可以梳理出完成作业的路线图。如果你需要帮助创建设计文档，我们可以提供帮助，官方提供了设计指南和Capers实验的设计文档示例。

八、评分标准

Gitlet有三个评分器：检查点评分器、完整评分器、Snaps评分器。

8.1 检查点评分

截止时间3月12日 23:59，占16分加分。提交到Gradescope上的“Project 2: Gitlet Checkpoint”自动判分器。测试内容包括：

- 程序可以编译。
- 通过骨架提供的样例测试： `testing/samples/*.in`，需要实现：
 - `init`
 - `add`
 - `commit`
 - `checkout -- [文件名]`
 - `checkout [提交ID] -- [文件名]`
 - `log`

此外会提示（但不评分）：是否通过风格检查（目前会忽略TODO类注释，最终提交时不会忽略），最终提交时会评分风格。3/4更新：允许有编译器警告。

你最多有1个提交token，每20分钟刷新一次。失败时不会提供完整日志（只会告诉你哪个测试失败，不会提供额外信息），但因为你有测试文件本身，可以在本地调试。

8.2 完整评分

截止时间4月2日 23:59，占1600分。完整评分器是更全面的测试集，你最多有1个token，token刷新频率如下：

- 2/20 - 3/19：每6小时一次
- 3/20 - 3/26：每3小时一次
- 3/26 - 4/2：每20分钟一次

和项目1一样，判分器访问有限，请你在开发过程中自己编写测试，不要过度依赖自动判分器检查代码。和检查点类似，完整判分器会提供每个测试功能的英文提示，但不会提供实际的.in文件。

8.3 Snaps提交评分

截止时间4月9日 23:59。你必须推送snaps仓库并提交到Snaps Gradescope作业，Gradescope分数才会同步到Beacon系统。推送snaps仓库执行以下命令：

代码块

```
1  cd $SNAPS_DIR
2  git push
3
```

推送snaps仓库后，有一个Gradescope作业需要你提交snaps-sp21-s***仓库（和项目1类似），仅针对完整评分（不包括检查点和加分项作业）。你可以在截止后一周内提交，如果超过一周忘记推送，需要使用迟交天数。

8.4 加分项

总共有 $16+32+64=112$ 分加分：

1. 检查点完成：16分
2. status命令输出“Modifications Not Staged For Commit”和“Untracked Files”板块：32分
3. 远程命令实现：64分

规范剩余部分是帮助你入门的资源，**测试/调试章节对你会非常有帮助**，本项目的测试和调试和之前项目不同，但并不复杂。

九、项目通用须知

前面讲解了大量命令，但不用担心，不同命令难度不同。每个命令标注了参考实现行数（只统计该命令专属代码，不统计多命令复用的代码），你不需要严格匹配行数，但可以大致估计每个命令的工作量。merge命令比其他命令代码量更大，不要留到最后写！

这是一个有挑战性的项目，如果你觉得无从下手是正常的。因此你可以比平时更深入地和同学协作，但需要遵守以下规则：

- 在gitlet/Main.java文件开头的注释中注明所有合作者。
- 不要分享具体代码，所有合作者必须独立写出自己的算法实现，保证代码存在差异。

Ed论坛上Gitlet的大讨论帖通常会很长，但里面有大量关于实现方法的优质讨论。本项目尤其建议你利用班级资源，在大讨论帖中寻找和你有类似问题的同学，你的问题不太可能是独有的（除非是和你设计相关的bug，这种情况请提交Gitbug）。

到这里规范已经提供了足够的信息让你开始项目，为了进一步帮助你，还有一些注意事项需要了解。

十、文件处理指南

本项目需要读写文件，你可能会发现 `java.io.File` 和 `java.nio.file.Files` 类很有用，实际上java.io和java.nio包中的各种工具都可能有帮助。请务必阅读我们提供的gitlet.Utils包，里面有很多我们写好的工具方法，深入研究这些方法会让本项目的IO部分简单很多！一个警告：如果你发现自己在使用Reader、Writer、Scanner或Stream，说明你把事情搞复杂了。

十一、序列化细节

你会发现Gitlet每次运行程序只能执行一条命令，为了让版本控制系统正常工作，你需要在命令之间记住提交树。这意味着你不仅需要设计运行时表示Gitlet内部结构的类，还需要在.gitlet目录中用文件做

对应持久化存储，跨多次程序运行保留数据。

如前所述，便捷的实现方式是将需要永久存储的运行时对象序列化到文件中，Java运行时会自动处理哪些字段需要转换为字节以及如何转换。你在实验6中已经做过序列化，这里不再重复，如果你对序列化还有疑问，请重新阅读实验6规范的对应部分并参考你自己的实验代码。

但有一个需要注意的坑：Java序列化会跟随指针。也就是说，不仅你传入writeObject的对象会被序列化写入，它指向的所有对象也会被序列化。比如如果你内部的commit表示用指针指向其他commit对象，那么写入分支头时会把整个提交子图中的所有commit（和blob）都写入一个文件，这通常不是你想要的。为了避免这个问题，运行时对象中不要用Java指针引用commit和blob，而是使用SHA-1哈希字符串，运行时维护这些字符串到对应运行时对象的映射，这个映射只在Gitlet运行时创建和填充，永远不要读写到文件中。

为了避免每次查找指针的开销，你可能希望同时保留（冗余的）commit指针和SHA-1字符串，你可以在对象中存储这些指针，同时通过声明为transient避免它们被写出，比如：

代码块

```
1 private transient MyCommitType parent1;  
2
```

这类字段不会被序列化，反序列化后会被设为默认值（引用类型为null）。读取包含transient字段的对象时，你需要手动将transient字段设置为正确的值。

遗憾的是，为了调试用文本编辑器查看程序生成的序列化文件是看不懂的，内容是Java私有的序列化编码。因此我们提供了一个简单的调试工具程序：`gitlet.DumpObj`，具体用法查看gitlet/DumpObj.java的JavaDoc注释。

十二、测试指南

建议完整阅读本节，也有配套视频可供参考。

和往常一样，测试是项目的一部分，请为每个命令提供自己的集成测试，覆盖所有规范功能，也可以自由添加单元测试。我们不提供单元测试，因为单元测试高度依赖你的实现。

我们提供了测试程序让编写集成测试更简单：`testing/tester.py`，它会解析后缀为.in的测试文件。你可以用以下命令运行所有测试：

代码块

```
1 make check  
2
```

如果需要查看失败测试的额外信息，比如程序输出，运行：

```
1 make check TESTER_FLAGS="--verbose"
2
```

如果要运行单个测试，进入testing子目录执行：

代码块

```
1 python3 tester.py --verbose FILE.in
2
```

其中FILE.in是你要检查的特定.in文件列表。**运行该命令时注意：**它不会重新编译你的代码，每次运行python命令前必须先执行make编译代码。

命令：

代码块

```
1 python3 tester.py --verbose --keep FILE.in
2
```

除了显示详细信息外，还会保留tester.py生成的目录，你可以在测试脚本检测到错误时查看目录中的文件。如果测试没有错误，目录也会保留，包含最终的所有内容。

实际上测试器实现了一个非常简单的领域专用语言（DSL），支持以下命令：

- 在测试目录创建或删除文件；
- 运行java gitlet.Main；
- 检查Gitlet输出是否匹配指定输出或描述可能输出的正则表达式；
- 检查文件存在、不存在以及文件内容。

执行命令（不带参数）：

代码块

```
1 python3 testing/tester.py
2
```

会显示该语言的完整说明。我们在testing/samples目录提供了一些示例，不要把你自己的测试放在该子目录，放在单独的位置避免和官方测试混淆（你的测试可能有bug）。将你所有的.in文件放在testing目录下名为student_tests的文件夹中，骨架中该文件夹初始为空。

我们在Makefile中添加了一些配置适配不同环境，如果你的系统调用Python3的命令是python，可以使用以下命令兼容运行：

```
代码块  
1 make PYTHON=python check  
2
```

你可以通过如下方式传递额外参数给tester.py：

```
代码块  
1 make TESTER_FLAGS="--keep --verbose"  
2
```

十三、使用官方参考实现测试

从2月28日周日开始，你可以使用官方参考实现验证你对命令的理解，也可以验证你自己的测试，使用指南见课程官网。

十四、集成测试详解

你提交Gitbug或来办公时间求助时，我们首先会要你提供失败的测试用例，因此学会写测试在本项目中至关重要。我们做了大量工作让写测试尽可能简单，请花时间阅读本节，理解提供的测试并自己写出合格的测试。

集成测试格式和Capers类似，如果你不知道Capers集成测试（.in文件）如何工作，请先阅读Capers规范对应章节。提供的测试覆盖并不全面，你肯定需要自己写测试才能拿到项目满分。要写测试，我们首先理解整个工作原理。

testing目录结构如下：

```
代码块
```

```

1  .
2  |— Makefile
3  |— student_tests <==== 你的.in文件放在这里
4  |— samples <==== 官方提供的.in样例
5  |   |— test01-init.in <==== 示例测试
6  |   |— test02-basic-checkout.in
7  |   |— test03-basic-log.in
8  |   |— test04-prev-checkout.in
9  |   └─ definitions.inc
10 |— src <==== 测试使用的文件
11 |   |— notwug.txt
12 |   └─ wug.txt
13 |— runner.py <==== 帮助调试程序的脚本
14 └─ tester.py <==== 测试程序的脚本
15

```

和Capers一样，这些测试会在testing目录中创建临时目录，执行.in文件中指定的命令。如果使用--keep标志，测试结束后临时目录会保留供你检查。

和Capers不同，我们需要处理工作目录中文件的内容，因此testing文件夹中有额外的src目录，该目录存储很多预填充内容的.txt文件，包含我们需要的特定内容。src存储实际文件内容，samples包含样例测试的.in文件（就是检查点测试）。你创建自己的测试时，应该添加到初始为空的student_tests文件夹。

Gitlet中的.in文件有更多功能，以下是tester.py文件中的命令说明：

代码块

```

1  # ... 注释, 无效果
2  I FILE 包含：将该语句替换为FILE的内容，相对于.in文件所在目录解析
3  C DIR 如果需要则创建名为DIR的子目录并切换到该目录下执行测试，如果DIR缺失则切回默认目录，
   该命令主要用于设置远程仓库
4  T N 将测试剩余部分中gitlet命令的超时设置为N秒
5  + NAME F 将src/F的内容复制到名为NAME的文件
6  - NAME 删除名为NAME的文件
7  > COMMAND OPERANDS
8  LINE1
9  LINE2
10 ...
11 <<<
12 运行gitlet.Main，参数为COMMAND ARGUMENTS，将输出和LINE1、LINE2等比较，如果有足够差异则报错。<<<分隔符后可以跟*，此时前面的行被视为Python正则表达式进行匹配。包含
   gitlet.Main程序的目录或JAR文件默认在--progdire指定的DIR中（默认为..）
13 = NAME F 检查名为NAME的文件和src/F内容完全相同，不同则报错
14 * NAME 检查名为NAME的文件不存在，存在则报错
15 E NAME 检查名为NAME的文件或目录存在，不存在则报错

```

```
16 D VAR "VALUE" 定义变量VAR的值为字面量VALUE，VALUE视为原始Python字符串（类似
    r"VALUE"），VALUE会先进行变量替换
17
```

不用担心上面提到的Python正则表达式，我们会展示它非常简单，还会通过示例讲解如何使用。

我们从头到尾走一遍一个测试，看看发生了什么，以test02-basic-checkout.in为例。

14.1 测试示例

第一次运行测试时，会创建一个初始为空的临时目录，目录结构如下：

代码块

```
1  .
2  |— Makefile
3  |— student_tests
4  |— samples
5  |   |— test01-init.in
6  |   |— test02-basic-checkout.in
7  |   |— test03-basic-log.in
8  |   |— test04-prev-checkout.in
9  |   └─ definitions.inc
10 |— src
11 |   |— notwug.txt
12 |   └─ wug.txt
13 |— test02-basic-checkout_0 <==== 刚创建
14 |— runner.py
15 └─ tester.py
16
```

这个临时目录就是本次测试使用的Gitlet仓库，我们会在里面添加内容、运行所有Gitlet命令。如果不删除目录第二次运行测试，会创建新目录test02-basic-checkout_1，以此类推。每次测试运行使用独立目录，不用担心测试之间互相干扰。

测试第一行是注释，直接忽略。下一部分是：

代码块

```
1  > init
2  <<<
3
```

从>行和<<<行之间没有内容可以看出，该命令没有输出。我们知道这会创建.gitlet文件夹，此时目录结构变为：

代码块

```
1  .
2  |— Makefile
3  |— student_tests
4  |— samples
5  |   |— test01-init.in
6  |   |— test02-basic-checkout.in
7  |   |— test03-basic-log.in
8  |   |— test04-prev-checkout.in
9  |   └─ definitions.inc
10 |— src
11 |   |— notwug.txt
12 |   └─ wug.txt
13 |— test02-basic-checkout_0
14 |   └─ .gitlet <==== 刚创建
15 |— runner.py
16 |— tester.py
17
```

下一部分是：

代码块

```
1  + wug.txt wug.txt
2
```

这行使用+命令，将右侧src目录中的文件内容复制到临时目录中左侧命名的文件（不存在则创建）。这里两个文件名碰巧相同，但没关系，因为它们在不同目录。执行该命令后目录结构变为：

代码块

```
1  .
2  |— Makefile
3  |— student_tests
4  |— samples
5  |   |— test01-init.in
6  |   |— test02-basic-checkout.in
7  |   |— test03-basic-log.in
8  |   |— test04-prev-checkout.in
9  |   └─ definitions.inc
10 |— src
11 |   |— notwug.txt
12 |   └─ wug.txt
13 |— test02-basic-checkout_0
14 |   └─ .gitlet
```

```
15 |   └─ wug.txt <==== 刚创建
16 |   └─ runner.py
17 |   └─ tester.py
18
```

现在我们知道src目录的作用了：它包含测试可以用来任意设置Gitlet仓库的文件内容。如果你想给文件添加特殊内容，应该将内容添加到src中合适命名的文件，然后使用这里的+命令。注意参数顺序不要搞反：右侧引用src目录中的文件，左侧引用临时目录中的文件。

下一部分是：

代码块

```
1 > add wug.txt
2 <<<
3
```

没有输出，此时wug.txt在临时目录中被标记为待添加暂存，此时test02-basic-checkout_0/.gitlet目录内部结构会变化，因为你需要持久化wug.txt被暂存的事实。

下一部分是：

代码块

```
1 > commit "added wug"
2 <<<
3
```

同样没有输出，.gitlet内部结构可能再次变化。

下一部分是：

代码块

```
1 + wug.txt notwug.txt
2
```

因为临时目录中已经存在wug.txt，它的内容会变为src/notwug.txt中的内容。

下一部分是：

代码块

```
1 > checkout -- wug.txt
2 <<<
```

同样没有输出，但它应该将临时目录中wug.txt的内容改回原始内容，也就是src/wug.txt的内容。下一个命令就是断言这一点：

代码块

```
1  = wug.txt wug.txt
2
```

这是一个断言：如果左侧文件（临时目录中的）内容和右侧文件（src目录中的）不完全相同，测试脚本会报错说文件内容不正确。

还有另外两个断言命令：

代码块

```
1  E NAME
2
```

断言临时目录中存在名为NAME的文件/文件夹，不检查内容，只检查存在性，不存在则测试失败。

代码块

```
1  * NAME
2
```

断言临时目录中不存在名为NAME的文件/文件夹，存在则测试失败。

这是测试的最后一行，测试结束。如果提供了--keep标志，临时目录会保留，否则会被删除。如果你怀疑.gitlet目录没有正确设置或者持久化有问题，可以保留目录查看。

14.2 测试初始化

你很快会发现，测试特定命令通常有大量重复的初始化步骤：比如测试checkout命令你需要：

1. 初始化Gitlet仓库
2. 创建一个提交，包含某个版本的文件（v1）
3. 创建另一个提交，包含另一个版本的该文件（v2）
4. 将文件检出到v1版本

如果要测试第二个提交未跟踪但第一个提交跟踪的文件，步骤会更多。

节省时间的是将所有初始化步骤放在一个文件中，使用I命令引入。比如我们将初始化步骤写在文件中：

代码块

```
1  # 初始化、添加并提交文件
2  > init
3  <<<
4  + a.txt wug.txt
5  > add a.txt
6  <<<
7  > commit "a is a wug"
8  <<<
9
```

我们将这个文件和其他测试一起放在samples目录，但文件后缀为.inc，比如命名为samples/commit_setup.inc。如果后缀是.in，测试脚本会误认为它是独立测试尝试单独运行。在实际测试中，我们只需要使用命令：

代码块

```
1  I commit_setup.inc
2
```

测试脚本会运行该文件中的所有命令，保留创建的临时目录，这样可以让测试保持简短易读。

官方包含了一个definitions.inc文件，预先设置了匹配模板方便使用，接下来我们了解什么是模板。

14.3 输出模板匹配

测试中最令人困惑的部分是log这类命令的输出，原因有几个：

1. 提交SHA会随着你修改代码、哈希更多内容而变化，你需要不断修改测试来适配SHA的变化。
2. 日期每次运行都会变化，因为时间只会向前走。
3. 会让测试变得非常长。

我们其实并不关心具体文本，只关心那里有合法SHA、日期格式正确，因此测试使用模板匹配。

你不需要理解底层概念，高层来说我们为某些文本定义模板（比如提交SHA），然后只检查输出符合该模板（不关心具体字母数字）。

以下是匹配log输出的示例：

代码块

```
1  # 首先从设置文件"导入"模板定义
```

```

2 I definitions.inc
3 # 这里添加创建对应提交信息的命令，示例中省略
4 > log
5 ===
6 ${COMMIT_HEAD}
7 added wug
8
9 ===
10 ${COMMIT_HEAD}
11 initial commit
12
13 <<<*
14

```

这部分和普通Gitlet命令类似，只是结尾是<<<*, 告诉测试脚本使用模板匹配。模板用\${PATTERN_NAME}包裹。

所有模板都定义在samples/definitions.inc中，你不需要理解实际模板，只需要知道它匹配什么。比如COMMIT_HEAD匹配提交头，格式类似：

代码块

```

1 commit fc26c386f550fc17a0d4d359d790bae33c47c54b
2

```

也就是任意提交SHA。创建测试预期输出时，你只需要知道log中有多少条目、提交信息是什么即可。

status命令也可以类似使用模板：

代码块

```

1 I definitions.inc
2 # 这里添加设置status的命令，示例中省略
3 > status
4 === Branches ===
5 \*master
6
7 === Staged Files ===
8 g.txt
9
10 === Removed Files ===
11
12 === Modifications Not Staged For Commit ===
13
14 === Untracked Files ===
15 ${ARBLINES}

```

```
16
17 <<<*
18
```

这里使用的ARBLINES模板匹配任意多行文本。如果你确实关心未跟踪文件内容，可以不用模板直接写死，但通常我们只关心g.txt被暂存待添加。

注意master分支前的*：status命令中HEAD分支前需要加*，如果使用模板匹配，预期输出中需要将*替换为*，原因超出课程范围，称为“转义”星号。如果不使用模板（命令结尾是<<<而不是<<<*），可以直接用*不需要加反斜杠。

模板最后一个功能是“保存”匹配到的部分。**警告**：这看起来像魔法，你不需要理解它如何工作，只需要知道有这个功能可用即可，你可以从官方提供的测试中复制相关部分，不需要从头写。我们通过test04-prev-checkout.in看看如何“捕获”或“保存”SHA：

代码块

```
1 I definitions.inc
2 # 每个${COMMIT_HEAD}会捕获对应提交的UID
3 # 这里省略创建多个带特定信息提交的设置步骤
4 > log
5 ===
6 ${COMMIT_HEAD}
7 version 2 of wug.txt
8
9 ===
10 ${COMMIT_HEAD}
11 version 1 of wug.txt
12
13 ===
14 ${COMMIT_HEAD}
15 initial commit
16
17 <<<*
18
```

log命令执行后会捕获UID（SHA），我们可以使用D命令将UID定义为变量：

代码块

```
1 # 第二个版本的UID
2 D UID2 "${1}"
3 # 第一个版本的UID
4 D UID1 "${2}"
5
```

注意编号是反的：编号从1开始，从log顶部开始计数，因此当前版本（第二个版本）定义为"\${1}"。我们不需要初始提交，因此不捕获它的UID。

现在我们就可以使用捕获的SHA执行checkout：

代码块

```
1 > checkout ${UID1} -- wug.txt
2 <<<
3
```

之后就可以添加断言确保checkout成功。

14.4 测试总结

测试脚本还有很多更复杂的功能，但以上内容足够写出高质量测试。你应该以官方提供的测试为示例入门，也可以在Ed上讨论测试设计的高层思路，你也可以分享自己的.in文件，但请确保分享前文件是正确的，并添加注释让其他同学和工作人员能看懂逻辑。

十五、集成测试调试

回忆实验6，新环境下集成测试调试略有不同。runner.py脚本的工作方式和Capers中完全一致，你应该阅读实验6规范中对应章节并观看链接的视频。这里描述调试策略：

15.1 定位出错的执行步骤

每个测试会多次运行你的程序，每次运行都有可能引入bug。首要任务是定位引入bug的那次程序执行。比如你测试status命令失败，输出只差一个文件：你说它是未跟踪的，但测试说它应该是暂存待添加。**这并不意味着status命令有bug**，有可能是add命令没有正确持久化文件被暂存的事实！这种情况下即使status命令完全正确，程序也会报错。

因此定位正确的（有bug的）程序执行非常重要：怎么做？使用runner.py脚本单步执行程序每次运行，每次执行后查看临时目录，确保所有内容都正确写入文件。序列化对象会比较难，因为我们知道它们的内容是无法识别的字节流：对于序列化对象，你只需要检查序列化时它们的内容是正确的即可，你甚至可能发现你根本没有序列化它！

最终你会找到bug。如果无法解决，可以来办公时间或提交Gitbug。注意：办公时间每个同学只有10分钟答疑时间，如果你的bug很复杂，预计需要TA超过10分钟解决，建议提交Gitbug并附上**尽可能详细的信息**，Gitbug信息越完整，回复越快。不要忘记更新设计文档：没有更新或完整设计文档的Gitbug会被拒绝。

十六、远程功能（加分项）

本项目核心是模仿Git的本地功能，这些功能可以帮助你备份自己的文件、维护多个版本。但Git真正的威力在于**远程**功能，支持和他人通过互联网协作：你和朋友可以在同一个代码库上协作，如果你修改了文件可以发送给朋友，反之亦然，你们都可以访问双方所有修改的共享历史。

完成加分项需要实现基础远程命令：add-remote、rm-remote、push、fetch、pull，完成后获得64分加分。**在完成项目其余部分前，不要尝试或规划加分项。**

根据你项目其余部分设计的灵活度，64分加分可能不值得投入对应的工作量，我们不要求所有人完成。我们的优先任务是帮助同学完成主项目；如果你做加分项，我们期望你比大多数同学更能独立解决问题。

16.1 远程命令详解

远程命令几点说明：

- 运行时间不计分，但也请不要实现明显不合理的逻辑。
- 所有命令相比真实Git对应命令做了大幅简化，因此和Git的具体差异通常不标注，但请注意存在差异。

16.1.1 add-remote

- **用法：**

代码块

```
1 java gitlet.Main add-remote [远程名] [远程目录名]/.gitlet
```

- **功能：**将给定的登录信息保存在给定远程名下，后续向该远程名push或pull时会使用该.gitlet目录。比如执行 `java gitlet.Main add-remote other ../testing/otherdir/.gitlet` 可以创建在所有位置都能运行的远程测试（在你的本地电脑或判分程序中都能工作）。这些命令中统一使用正斜杠，让程序将所有正斜杠转换为路径分隔符（Unix是正斜杠，Windows是反斜杠），Java提供了类变量 `java.io.File.separator` 表示该字符。
- **失败场景：**同名远程已存在，输出错误：

代码块

```
1 A remote with that name already exists.
```

- **危险命令？：**否

16.1.2 rm-remote

- **用法：**

代码块
`java gitlet.Main rm-remote [远程名]`

- **功能：**删除和给定远程名关联的信息，如果你想修改之前添加的远程，需要先删除再重新添加。
- **失败场景：**给定名称的远程不存在，输出错误：

代码块

```
1 A remote with that name does not exist.
```

- **危险命令？：**否

16.1.3 push

- **用法：**

代码块

```
1 java gitlet.Main push [远程名] [远程分支名]
```

- **功能：**尝试将当前分支的提交追加到给定远程的指定分支末尾。细节：该命令仅当远程分支的头在当前本地头的历史中时才能工作，意味着本地分支包含远程分支未来的一些提交。此时将未来的提交追加到远程分支，然后远程分支重置到追加提交的头部（因此它的头和本地头相同），这称为快进（fast-forwarding）。如果远程机器上的Gitlet系统存在但没有输入分支，直接在远程Gitlet中添加该分支。
- **失败场景：**如果远程分支的头不在当前本地头的历史中，输出错误：

代码块

```
1 Please pull down remote changes before pushing.
```

代码块

```
1 Remote directory not found.
```

- **危险命令？：**否

16.1.4 fetch

- **用法：**

```
1 java gitlet.Main fetch [远程名] [远程分支名]
```

- **功能：**将远程Gitlet仓库的提交下载到本地Gitlet仓库。本质上是将远程仓库中指定分支的所有提交和blob（当前仓库没有的）复制到本地.gitlet中名为 `[远程名]/[远程分支名]` 的分支（和真实Git一样），将 `[远程名]/[远程分支名]` 指向头提交（从而将远程仓库分支内容复制到本地）。如果本地之前不存在该分支则创建。
- **失败场景：**如果远程Gitlet仓库没有给定分支名，输出错误：

代码块

```
1 That remote does not have that branch.
```

代码块

```
1 Remote directory not found.
```

- **危险命令？：**否

16.1.5 pull

- **用法：**

代码块

```
1 java gitlet.Main pull [远程名] [远程分支名]
```

- **功能：**和fetch命令一样获取分支 `[远程名]/[远程分支名]`，然后将获取的分支合并到当前分支。
- **失败场景：**就是fetch和merge失败场景的并集。
- **危险命令？：**是

十七、开发避坑清单

经验表明以下做法会给你带来无尽的痛苦，导致程序无法工作、出现难以复现的bug（海森堡bug）：

1. 因为你可能会在文件中存储各种信息（比如提交），你可能会 tempted to 使用看起来方便的文件系统操作（比如列目录）来遍历所有文件。请小心：`File.list` 和 `File.listFiles` 方法返回文件名的顺序是不确定的，如果你用它们实现log命令，会得到随机结果。

2. Windows用户尤其需要注意：Unix（或MacOS）的文件分隔符是/，Windows是\。如果你在程序中通过拼接目录名和文件名，显式使用/或\，肯定会在某个系统上无法工作。Java提供了系统相关的文件分隔符（`System.getProperty("file.separator")`），或者你可以使用File的多参数构造函数。
3. 序列化时使用HashMap要小心！HashMap中元素的顺序是不确定的，解决方案是使用TreeMap，它永远保持固定顺序。

十八、致谢

感谢Alicia Luengo、Josh Hug、Sarah Kim、Austin Chen、Andrew Huang、Yan Zhao、Matthew Chow，特别感谢Alan Yao、Daniel Nguyen和Armani Ferrante为本项目提供反馈。感谢Git这个优秀的工具。

本项目很大程度上灵感来自Philip Nilsson的优秀文章。

本项目由Joseph Moghadam创建，Paul Hilfinger为2015秋、2017秋、2019秋版本做了修改。

（注：部分内容可能由 AI 生成）